



Autonomous Mass-Aware Sorting System

#Team 8 Armstrong

Team Member: Wynn Huo
Han Wu
Michael Xiong
Lisha Yuan

UCLA Samueli
School of Engineering

Overview



Improper stacking may cause damage (heavy items crushing lighter ones)



Build an Autonomous Mass-Aware, Scale Less Sorting System



Control Objective:

- Integrate weight measurement and sorting into a single efficient workflow.
- Enable adaptive grasping by adjusting gripping force based on item weight
- Case Study: find the best way to estimate weight.

Objectives of controller design



- **Estimators**
 - **Estimate weight accurately**
- **Trajectory Tracking**
 - **Moving stuff with different weights along the trajectory**
- **Estimator + Trajectory Tracking**
 - **Range Tradeoff**
 - **Precision and accuracy Tradeoff**

Methods - 4 Estimators



sPID

$$\hat{m} = (\bar{\tau}_{\text{loaded}} - \bar{\tau}_{\text{empty}}) / (g \cdot d)$$

- Disable gravity compensation at the elbow only.
- The controller's holding torque becomes a scale.
- Calibrates baseline $\bar{\tau}_{\text{empty}}$
- Weighs at rest

Lyapunov

$$\hat{m} = (2 \cdot \Delta E_{\text{spring}}) / (g \cdot \Delta h)$$

- With $k_i = 0$ each joint is a spring.
- The payload sags the arm, and the energy balance returns the mass.
- Calibrates empty pose (q, z)
- Weighs at rest

Momentum

$$\hat{m} = - \langle r - r_{\text{empty}}, J_z \rangle / (g \cdot \|J_z\|^2)$$

- Compare actual vs. predicted generalized momentum.
- The residual converges to the payload torque, so no noisy $\ddot{q}$ needed.
- Calibrates r_{empty}
- Weighs in motion

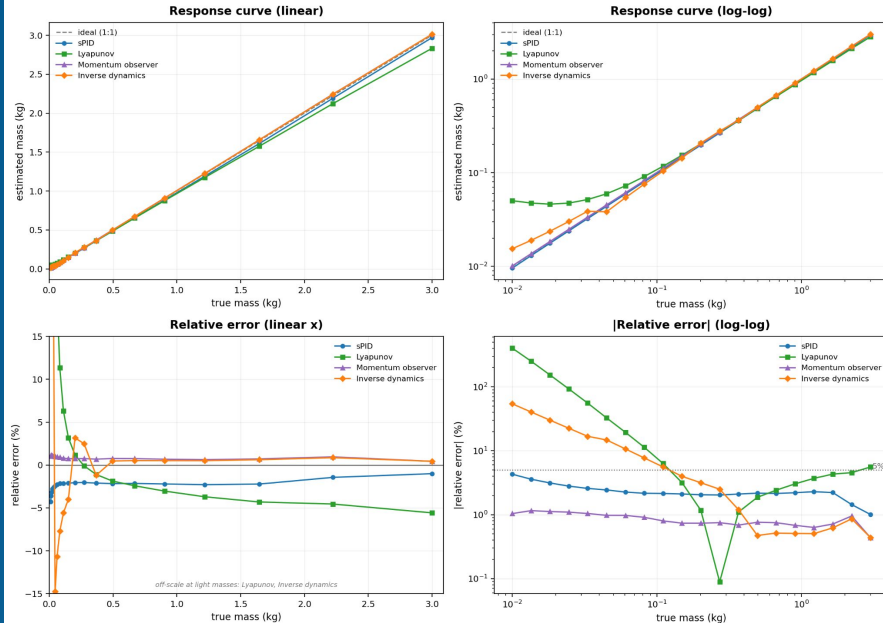
Inverse Dynamics

$$\tau_{\text{resid}} \approx m J_v(q)^T \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix}, \quad \hat{m} = \frac{a(q)^T \tau_{\text{resid}}}{a(q)^T a(q)}$$

- Subtract the arm's known dynamics from measured torque
- What remains is the payload's gravity pull, fit by least squares.
- Need no calibration
- Weighs during motion

Results

Estimator Comparison with PID Controller



Estimation accuracy across 10 g - 3 kg sweep

Estimator	Mean err	Mean err	RMSE
sPID	-2.4%	2.4%	2.5%
Lyapunov	+50.2%	52.8%	114.5%
Momentum observer	+0.8%	0.8%	0.9%
Inverse dynamics	+6.5%	10.9%	18.3%

Momentum observer is most accurate (0.9% RMSE)

- Only method with both an empty-arm baseline and all-six-joint residuals.

Every estimator has an absolute error floor, not a percentage error

- Each method mis-weighs by a fixed number of grams (slope ≈ -1), so the light end is where methods separate.

Calibration trades convenience for light-mass accuracy

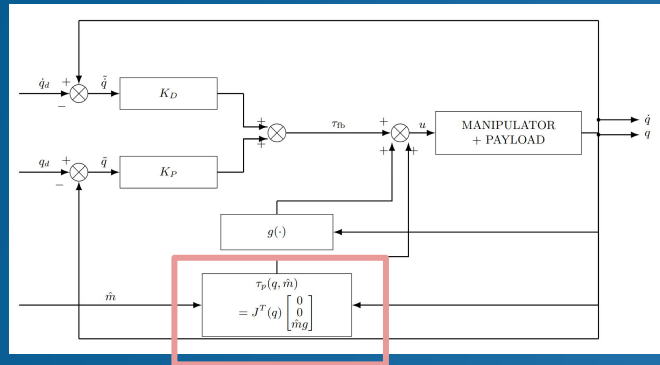
- Calibrated methods cancel gravity-model error with an empty-arm baseline. ID carries that bias, causing 18% overall RMSE.

Lyapunov can't measure light payloads (114% RMSE)

- Spring sag scales with payload mass; below ~ 100 g it's buried in noise.
- $\hat{m} = 2\Delta E / (g \cdot \Delta h)$: for light payloads both ΔE and Δh go to zero.

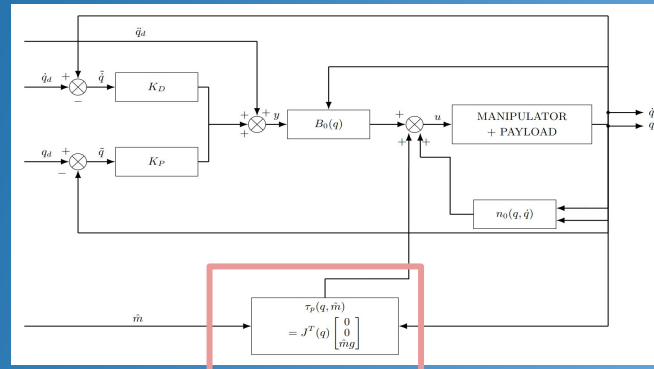
Methods - Trajectory Tracking Controllers

- PD with gravity control



payload compensation

- Inverse dynamics control



payload compensation

- **Controller design requirements:**

- Tracking accuracy: joint position error ≤ 2 deg
- Torque feasibility: commanded torque $\leq 0.95 \cdot$ actuator torque limit

- **Improvement on controller design:**

- add a **payload compensation term** after getting the mass from estimator

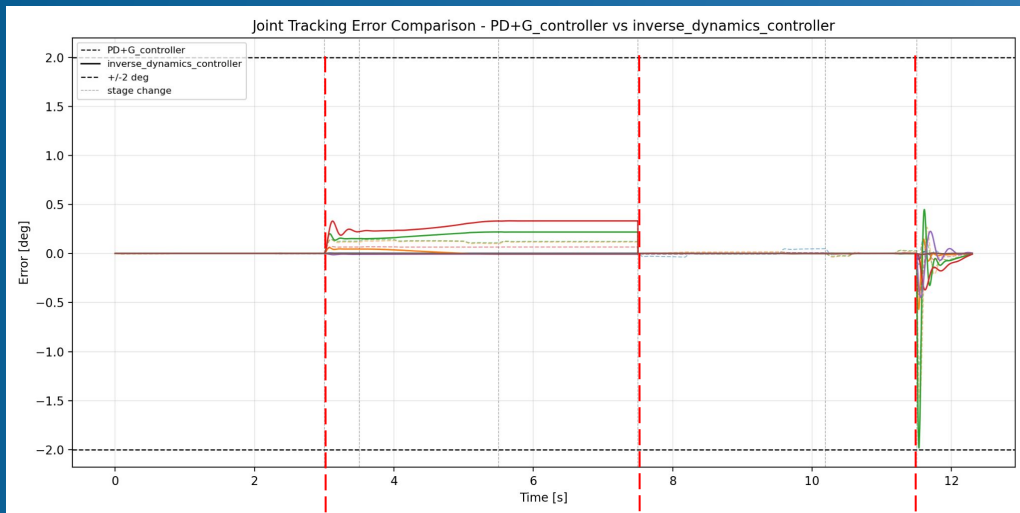
The inverse dynamics controller assumes that the MuJoCo robot model provides a sufficiently accurate robot mass matrix and bias terms. Payload compensation is simplified as payload gravity compensation only.

Results - Controllers performance

- Effect of Payload Compensation on Tracking Performance

payload = 1.5kg

avg EE speed = 0.30m/s



move to grasp point

lift to weigh point

add payload
compensation

grasp

weigh (estimator)

release

Controller parameters tuning results:

PD controller:

$k_p = [3150.0, 3850.0, 3150.0, 1260.0, 980.0, 700.0]$

$k_d = [78.4, 89.6, 78.4, 39.2, 33.6, 28.0]$

$k_i = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]$

ID controller:

$k_p = [1800.0, 2200.0, 1800.0, 720.0, 560.0, 400.0]$

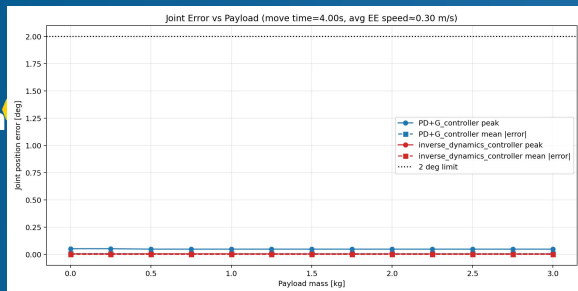
$k_d = [28.0, 32.0, 28.0, 14.0, 12.0, 10.0]$

Results - Controllers performance

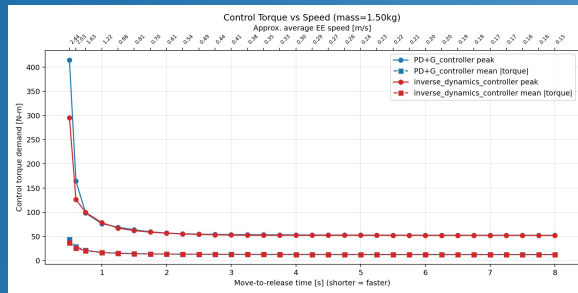
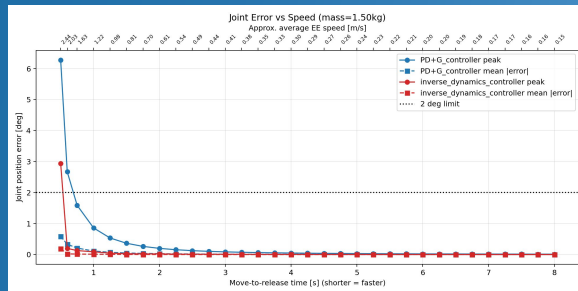
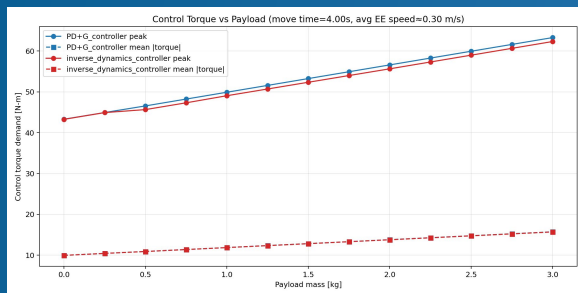
• Payload Robustness

• Speed Robustness

Joint position error



Control torque



➤ Both controllers maintain stable tracking under varying payloads.

➤ **PD+G controller** remains effective for moderate and slow motions but **degrades faster** as speed increases.

➤ **Inverse dynamics controller** shows stronger robustness at high trajectory speeds.

The comparison interval is the period when payload compensation is applied.

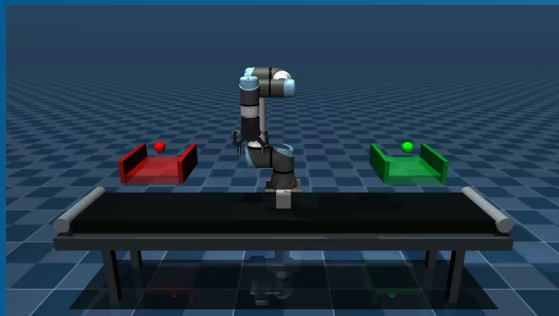
During the comparison, the PD gains of both controllers are kept unchanged.

MuJoCo does not allow exactly zero body mass, so the 0 kg case is simulated using a very small positive mass internally.

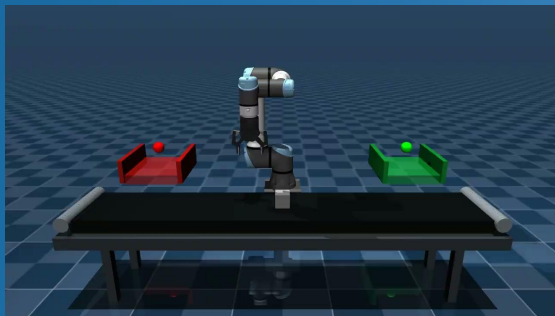
For controller performance comparison, true payload mass is used. This isolates controller behavior from estimator error.

Integration Demo (controller+estimator)

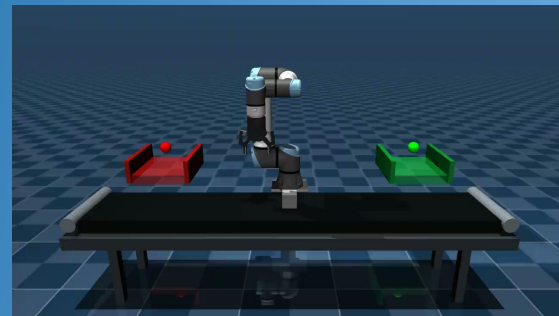
PID+sPID



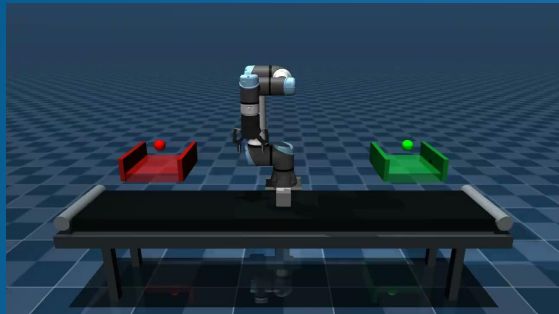
PID+ ID



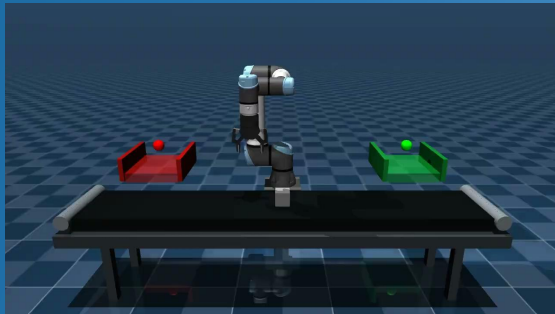
PID+ Momentum



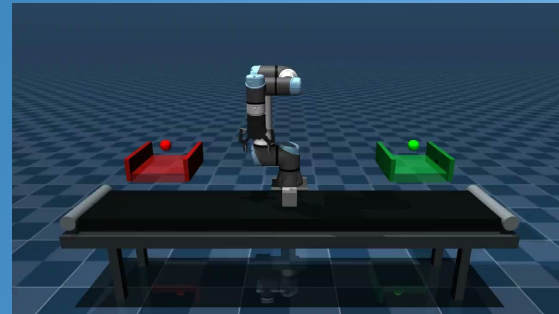
ID+sPID



ID+ID



ID+Momentum

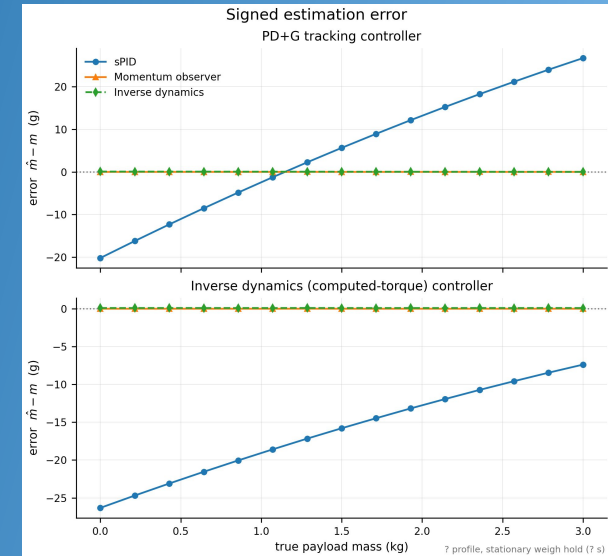
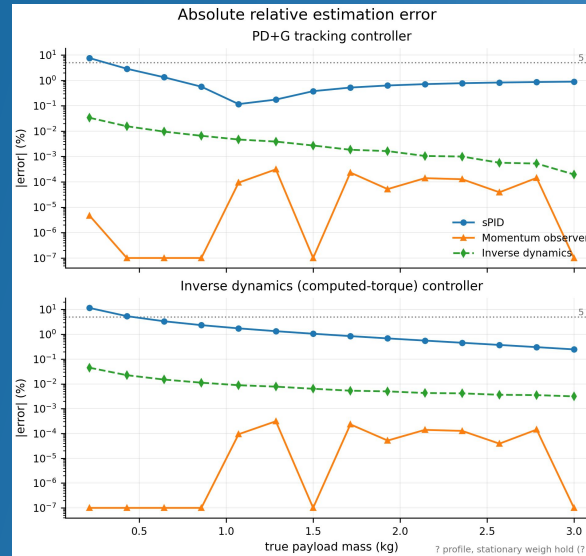
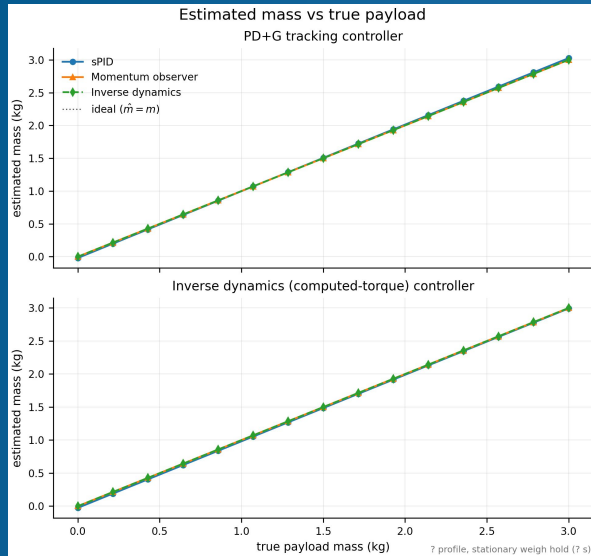


Does the Best Tracking Controller Produce the Best Payload Estimate?

Assumption: Controller-estimator pair with similar principles has better estimation accuracy.

Results - Integration performance

- We compare performances of different estimators in different trajectory controller
 - All estimators shows decent performance in general
 - **Inverse-dynamics** estimator and **Momentum-observer** estimator shows best accuracy in both controllers
 - **sPID-Error** estimator shows relatively bad performance under both controllers



Assessment of the results



- **Residual-based estimators are robust**
 - ID and momentum-based estimators use dynamics residuals induced by the payload, instead of relying on controller error.
- **Stable weighing reduces controller influence**
 - At a quasi-static hold, tracking transients fade and the payload gravity contribution becomes clearer in the residual.
- **PID-error estimator is less reliable**
 - It uses a single-joint PID torque/error signal, which is indirect and controller-dependent.
 - The selected joint torque mixes payload effect with tracking error, settling transient, gain response, gravity mismatch, and joint coupling.
- In general, **ID+Momentum** or **ID + ID** are the best combination for our task considering trajectory tracking performance

Discussion



Comparing the different estimators:

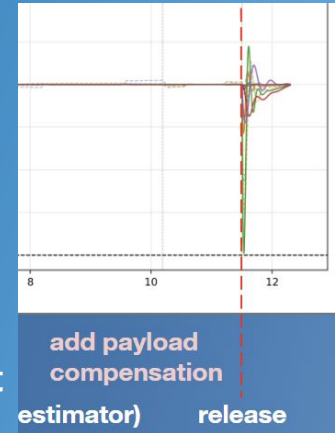
- All estimator except Lyapunov have proper ability to estimate the weight

Comparing the different trajectory controller:

- Both PD+ G Controller and Inverse Dynamic Controller have strong ability tracking the trajectory
- Inverse Dynamic Controller has stronger robustness

Boundary Conditions

- When the mass is light, error and oscillation occur. Typical case could be when the mass is released.



Discussion



Limitations

- Payload range only covers 0–5 kg
- Simulation uses ideal sensing and model information
- Gripper-payload interaction is simplified

Marginal Effect

- Tradeoff between the range of weight and the performance

Future Work



- Expand the payload range beyond 0--5 kg to better compare estimator robustness.
- Improve gripper-payload modeling by considering contact compliance, grasp offset, and possible object slip.
- Decouple trajectory tracking from weighing by designing a dedicated weight-estimation controller.
- Optimize the estimation pipeline, including weighing pose, hold duration, sample rejection, and multi-joint estimation.



Thank You!